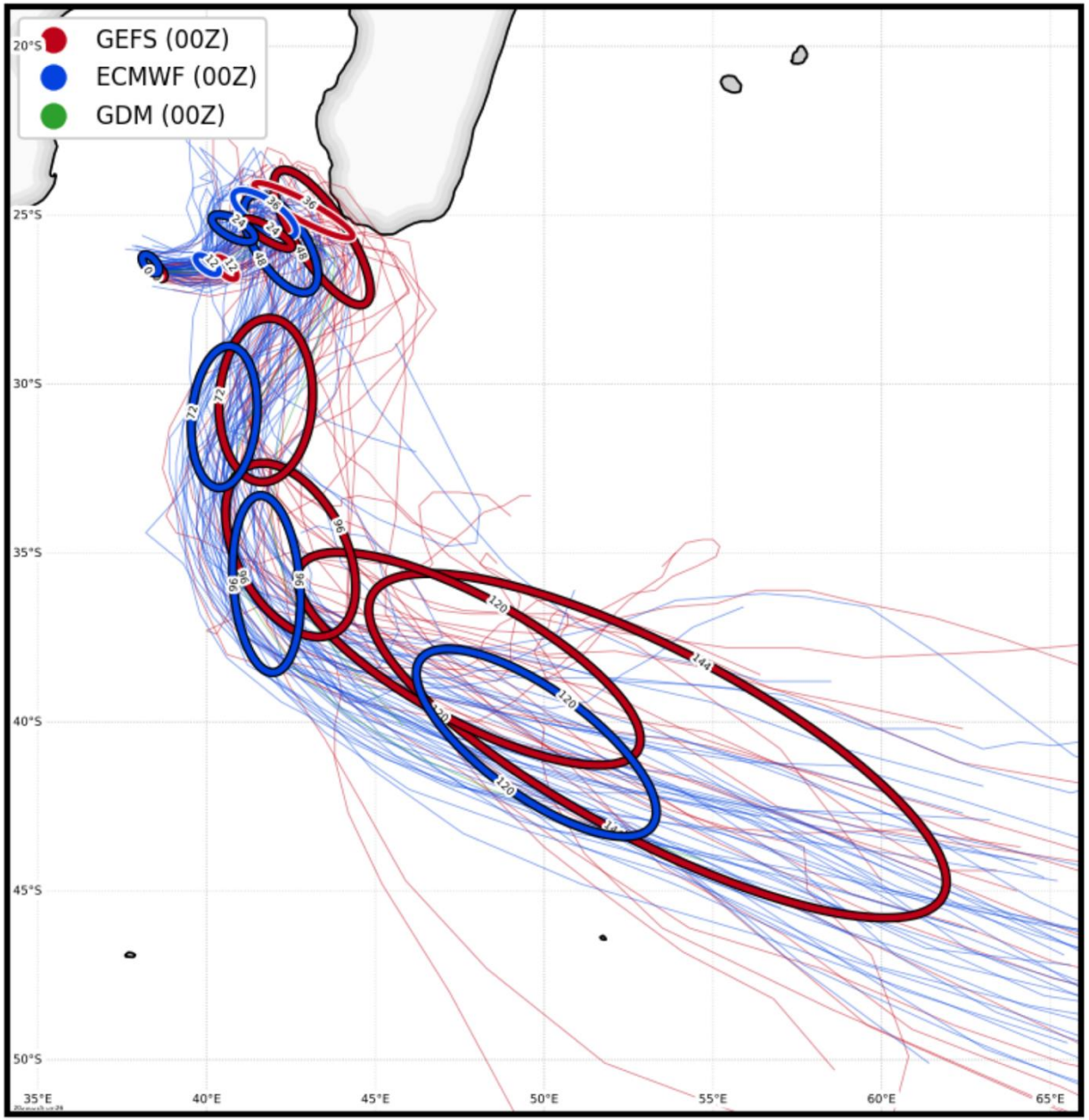
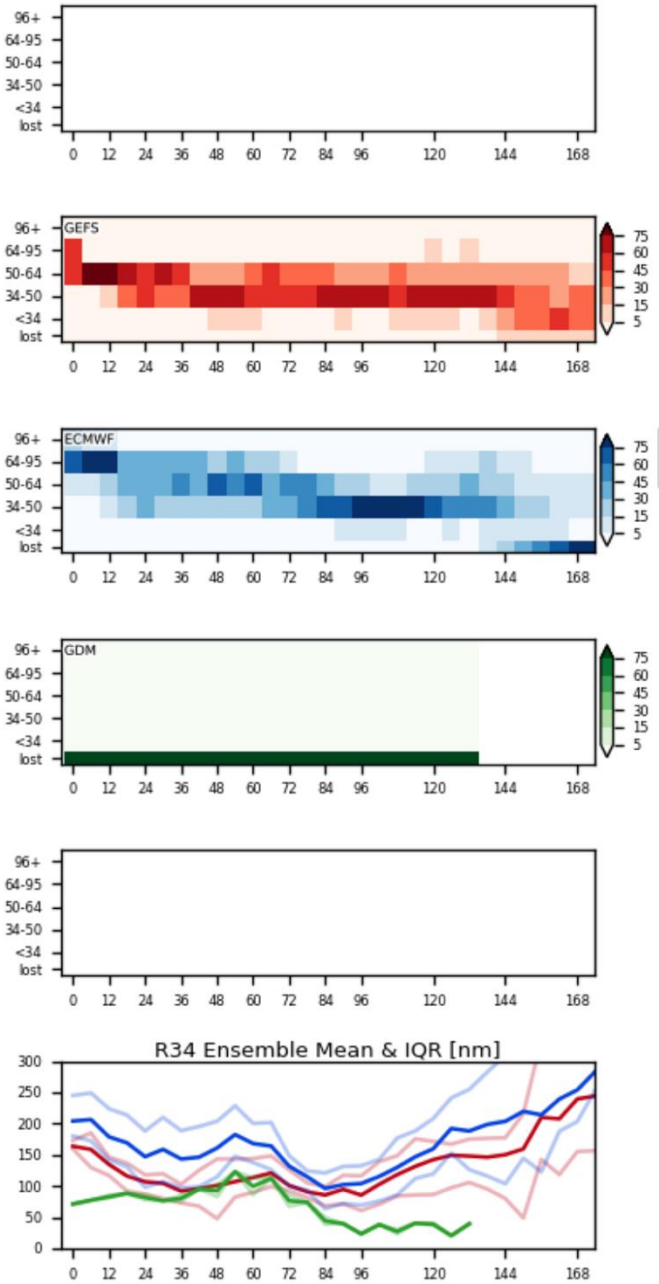
Module 4: Probabilistic/Ensemble Forecasting

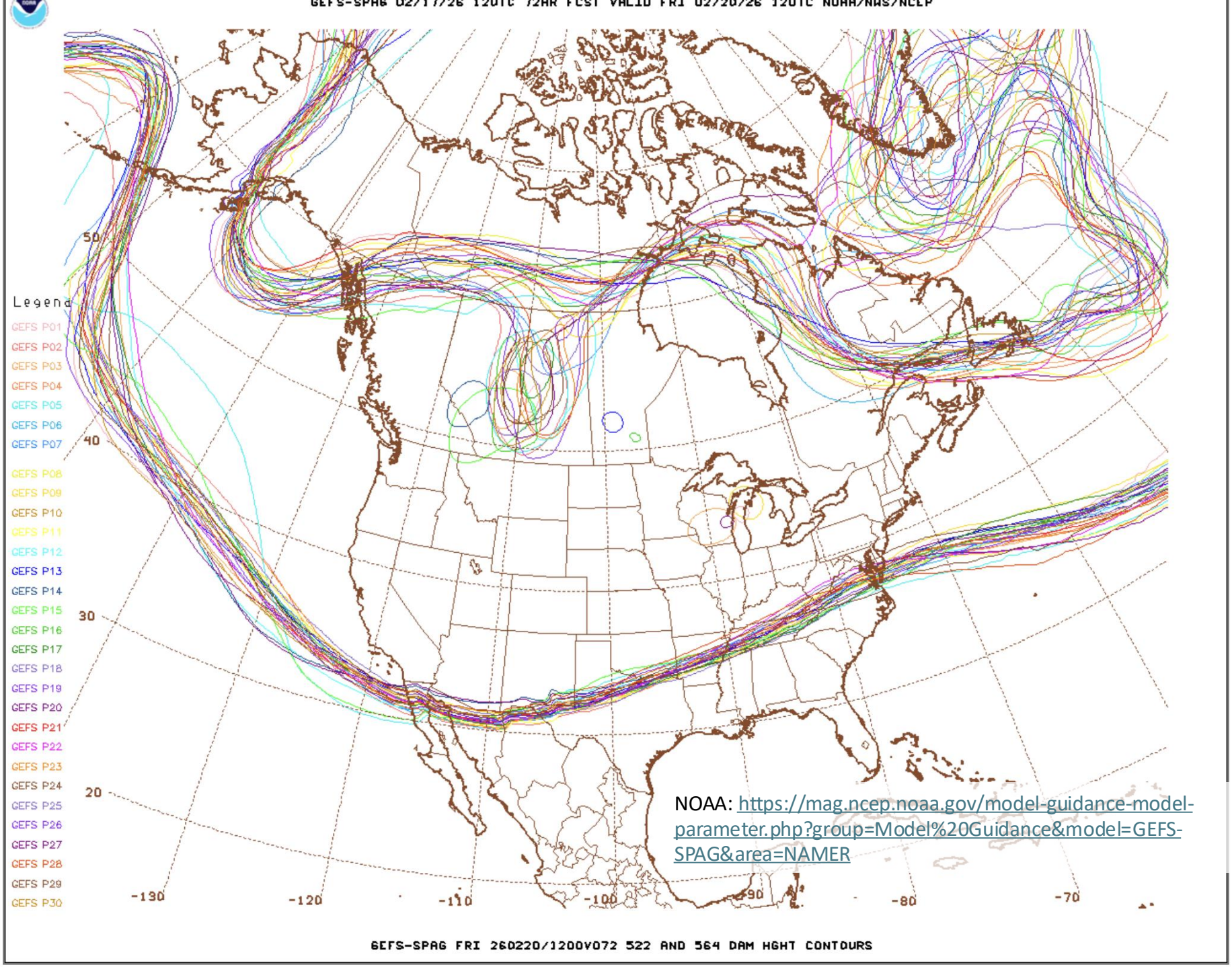
SH212026 -- 20260215 00Z



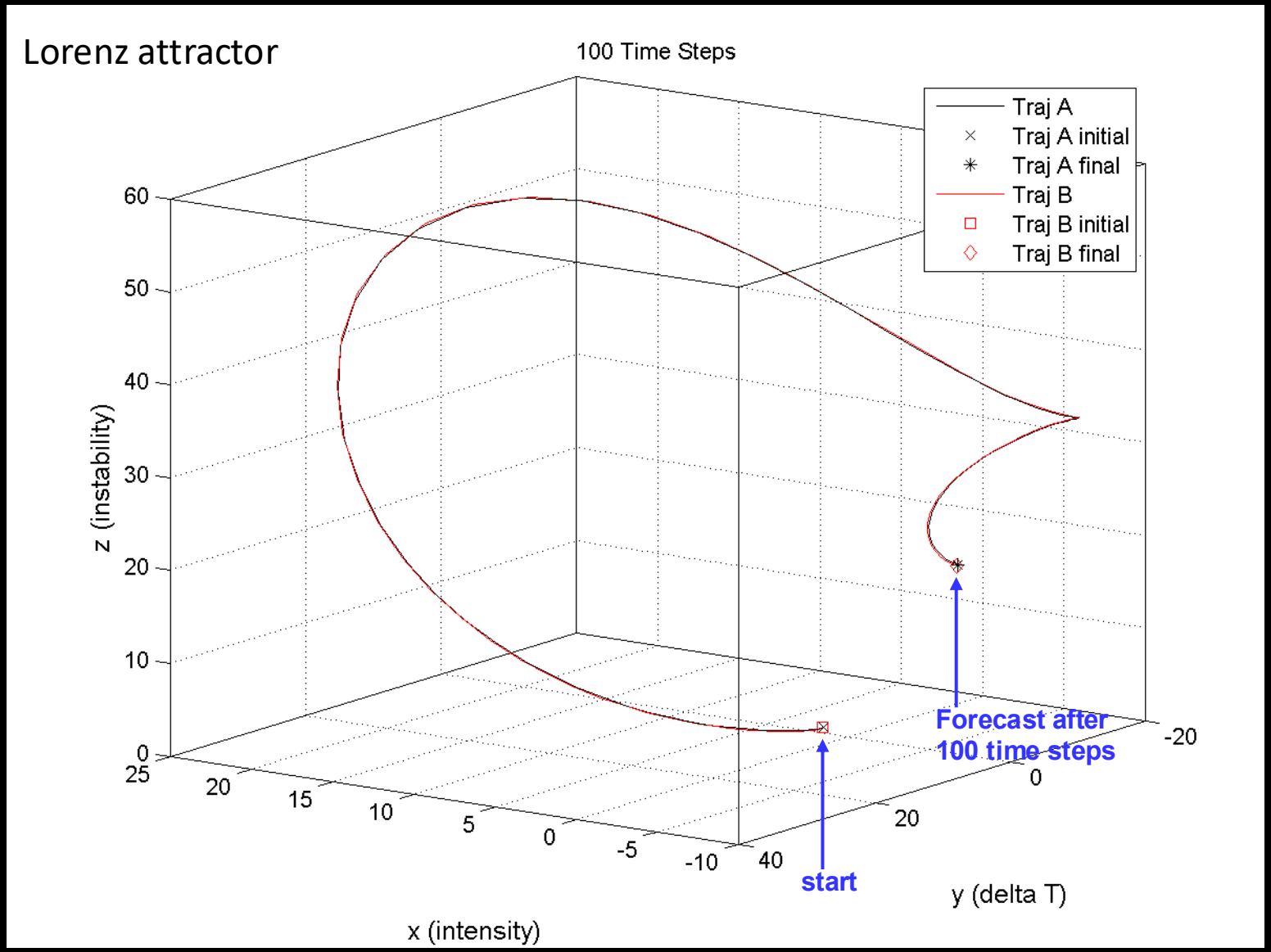
Ensemble Intensity Distributions [%]



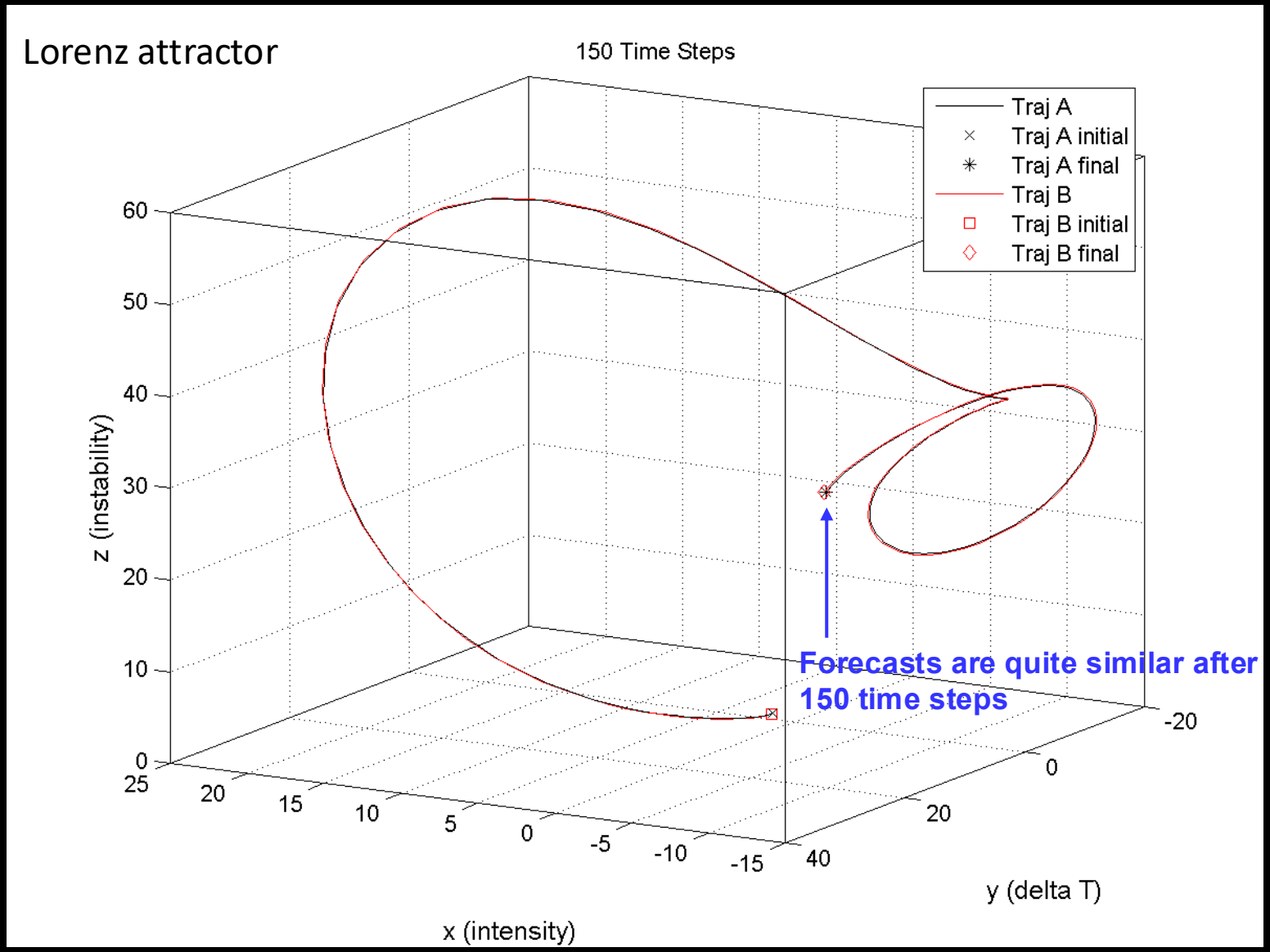
For model diagnostic purposes. Global ensembles are generally not reliable for intensity or size. Ellipses include 1 σ of spread.



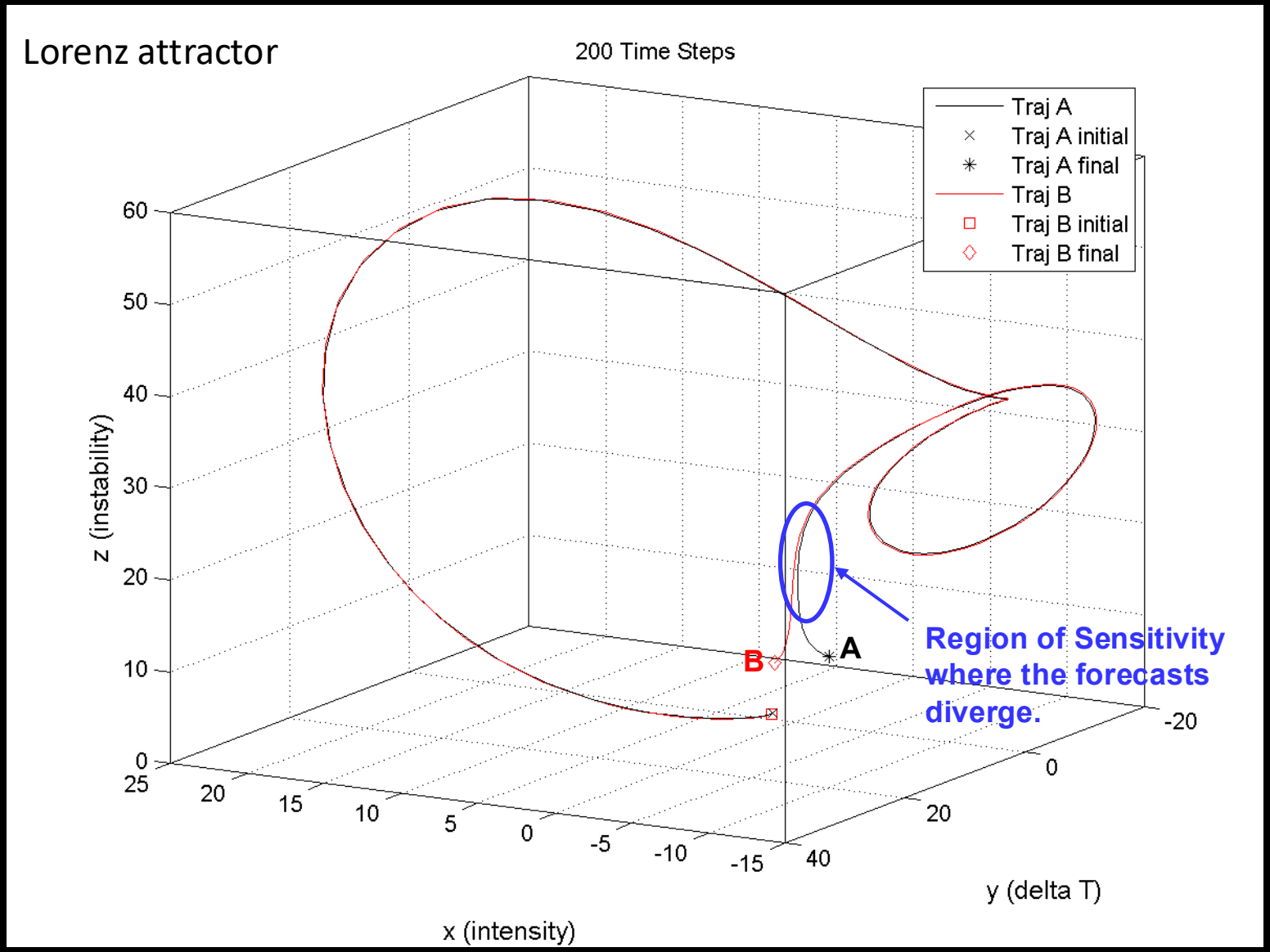
NOAA: https://mag.ncep.noaa.gov/model-guidance-model-parameter.php?group=Model%20Guidance&model=GEFS-SPAG&area=NAMER



Trajectory B starts with slightly different initial conditions than **Trajectory A**

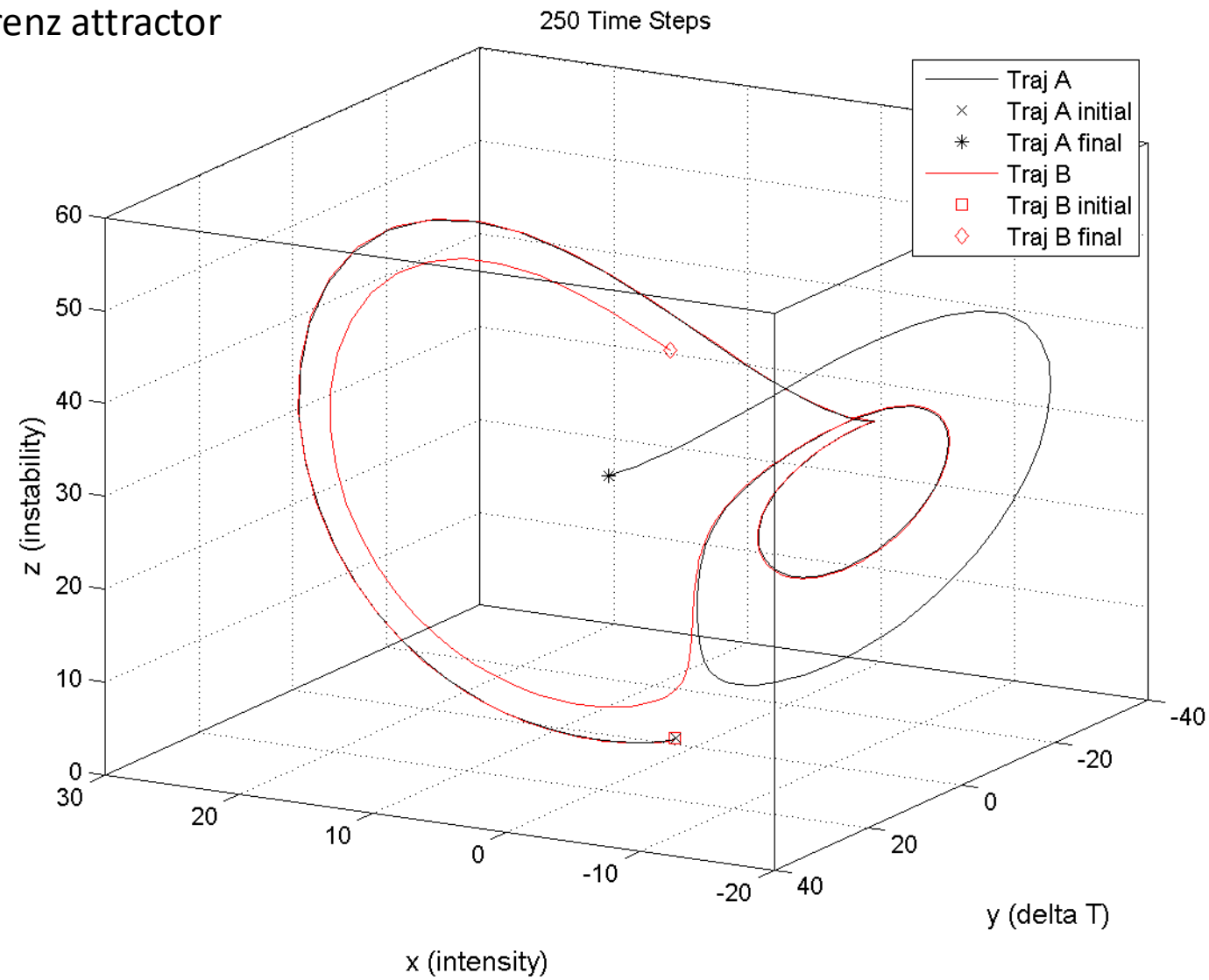


Trajectory B and **Trajectory A** have similar forecasts.

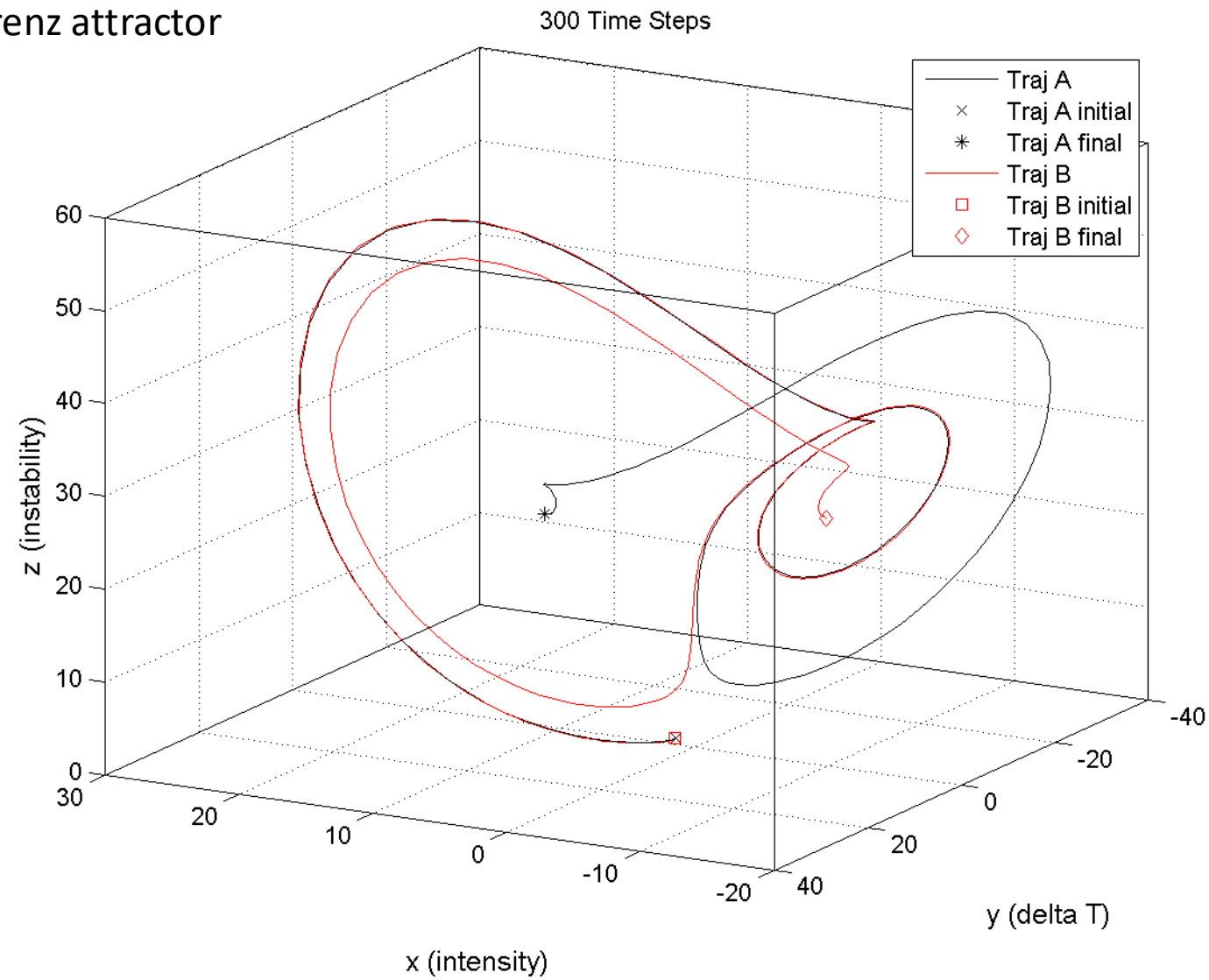


Trajectory B and **Trajectory A** forecasts are now quite different!

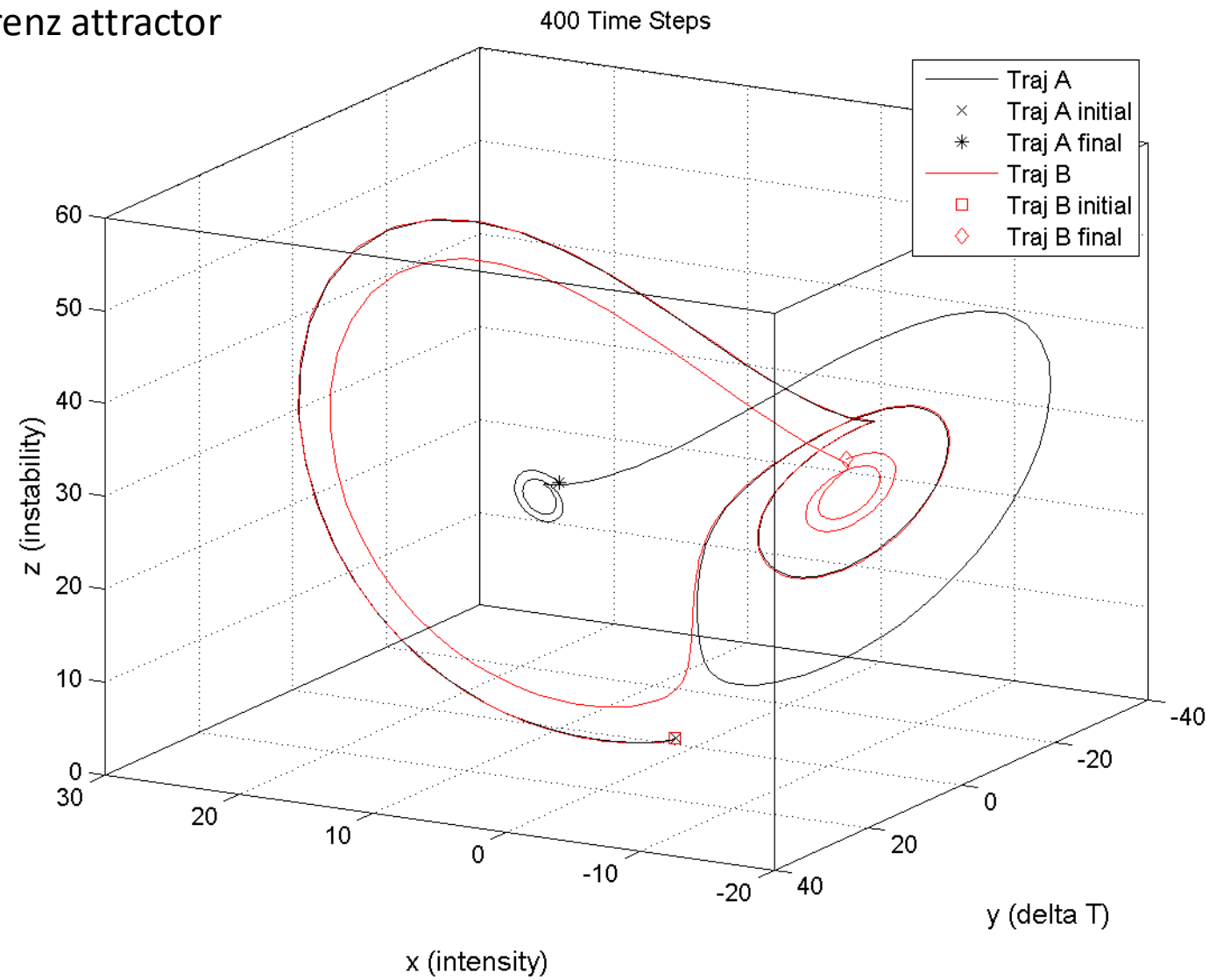
Lorenz attractor



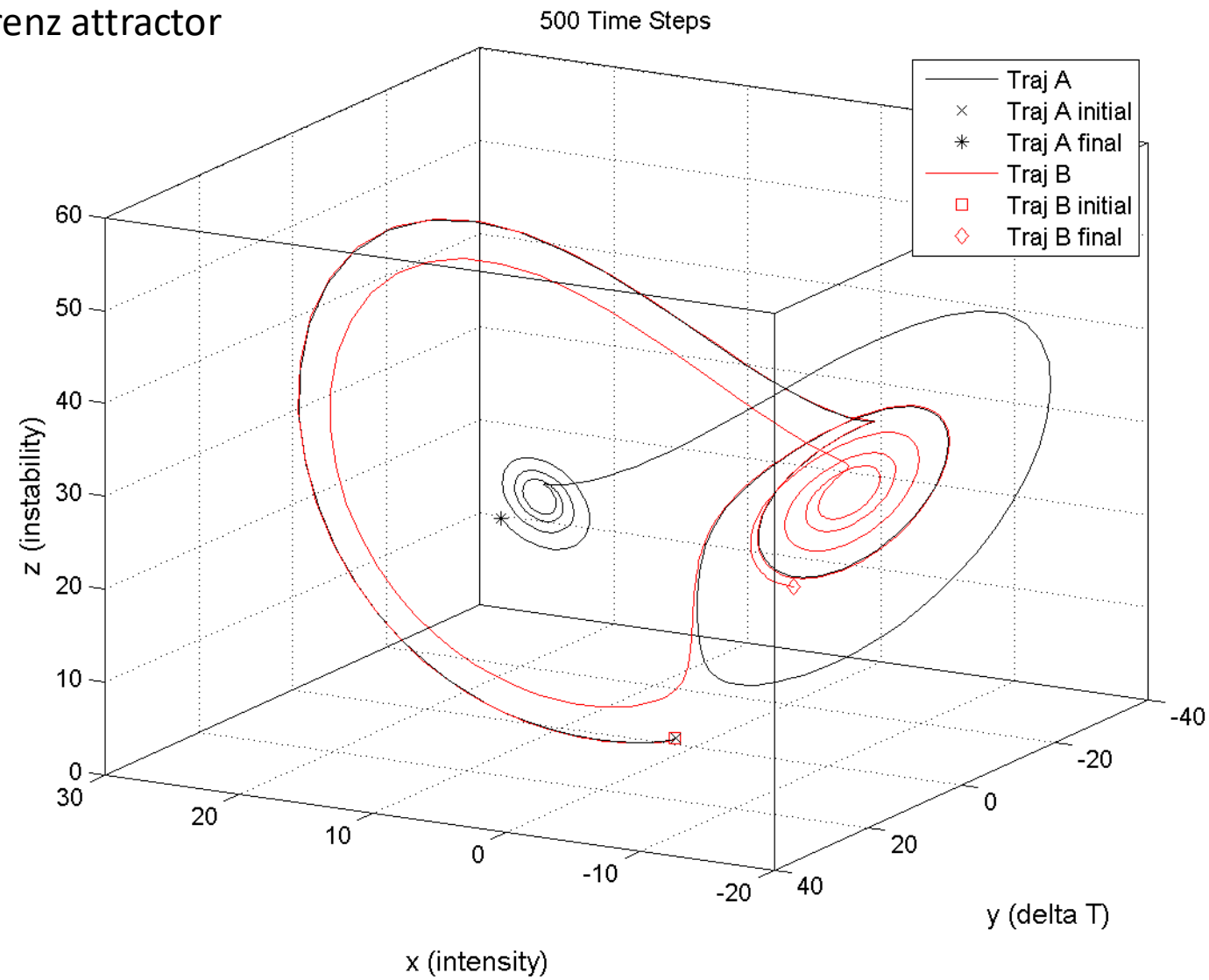
Lorenz attractor



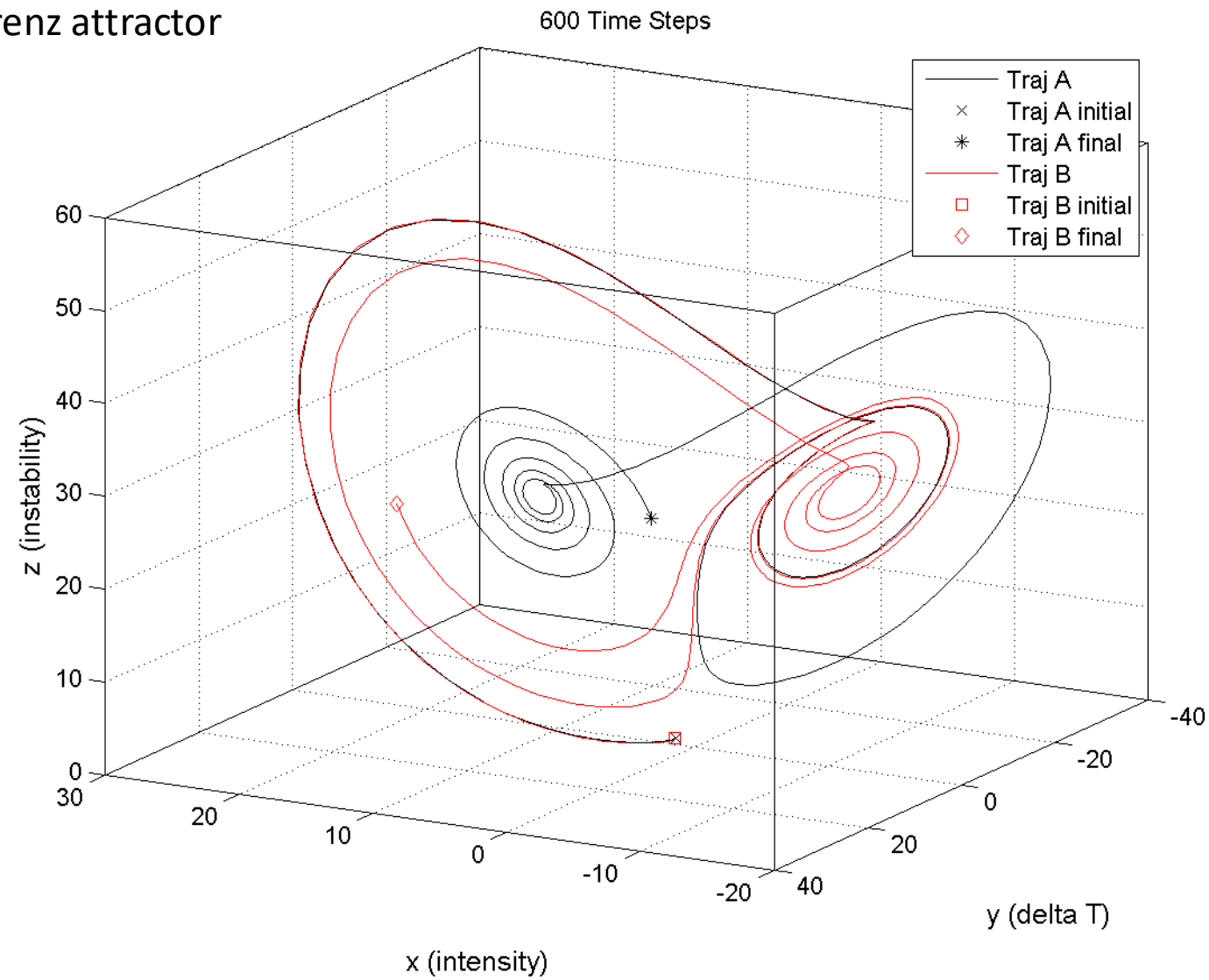
Lorenz attractor



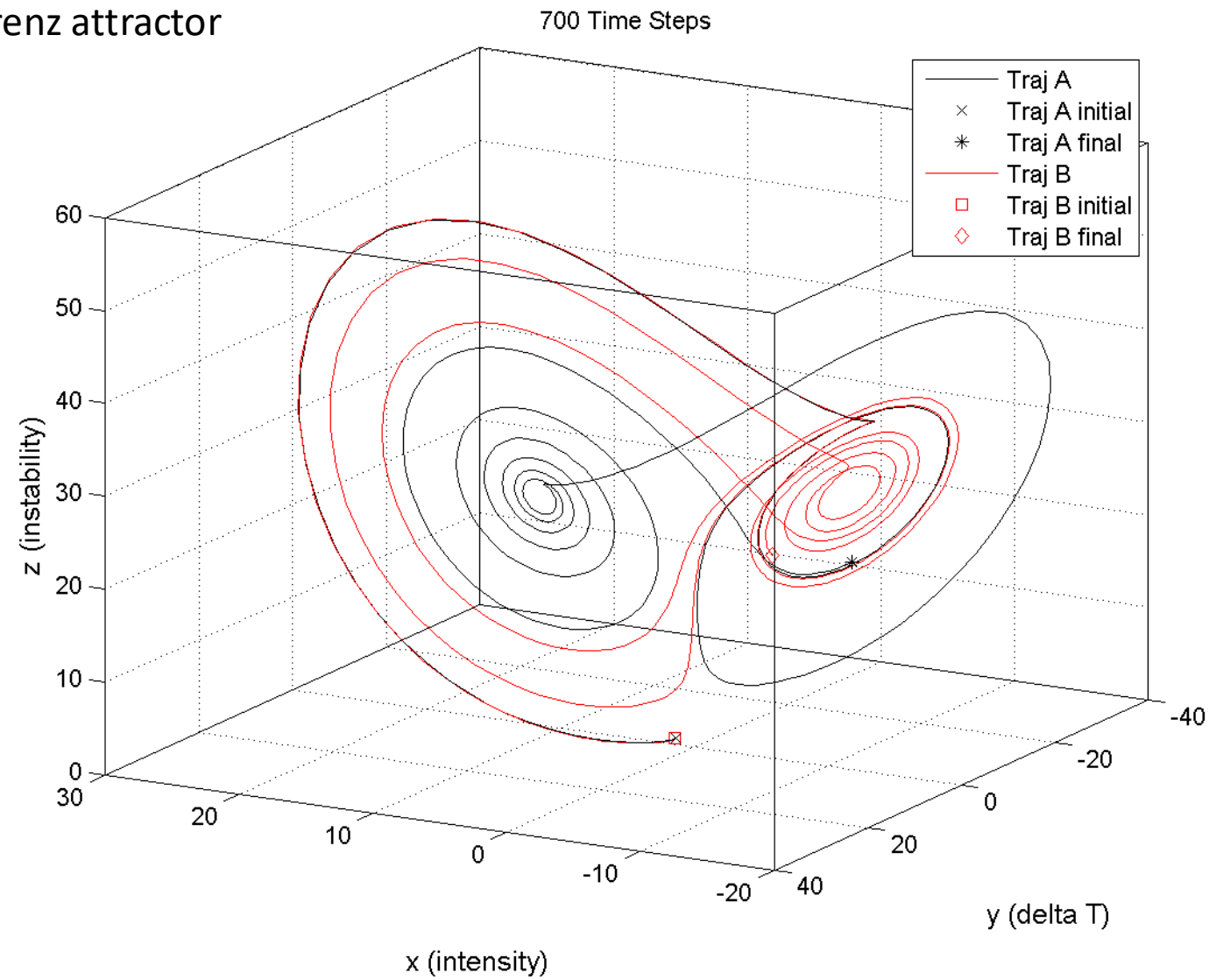
Lorenz attractor



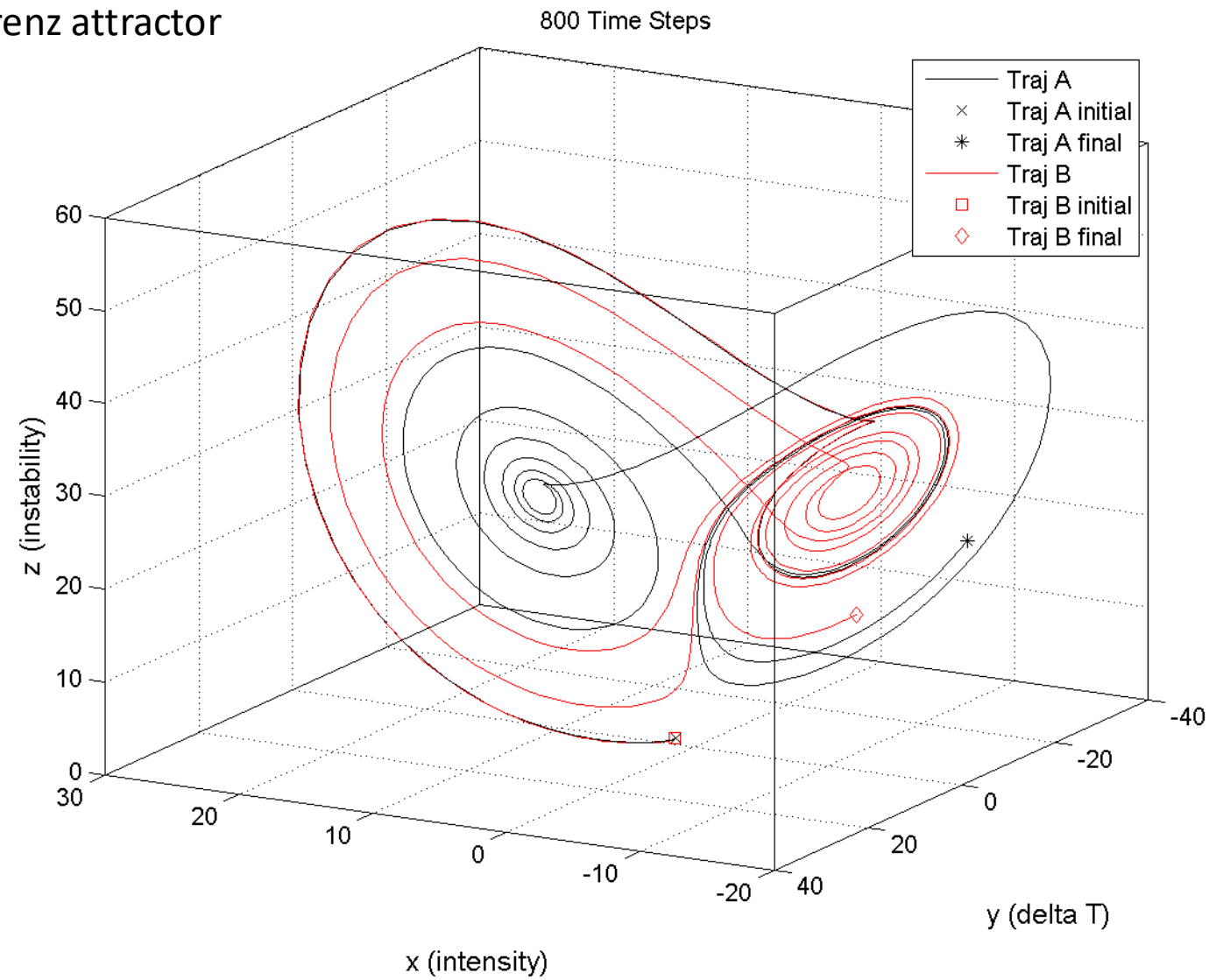
Lorenz attractor



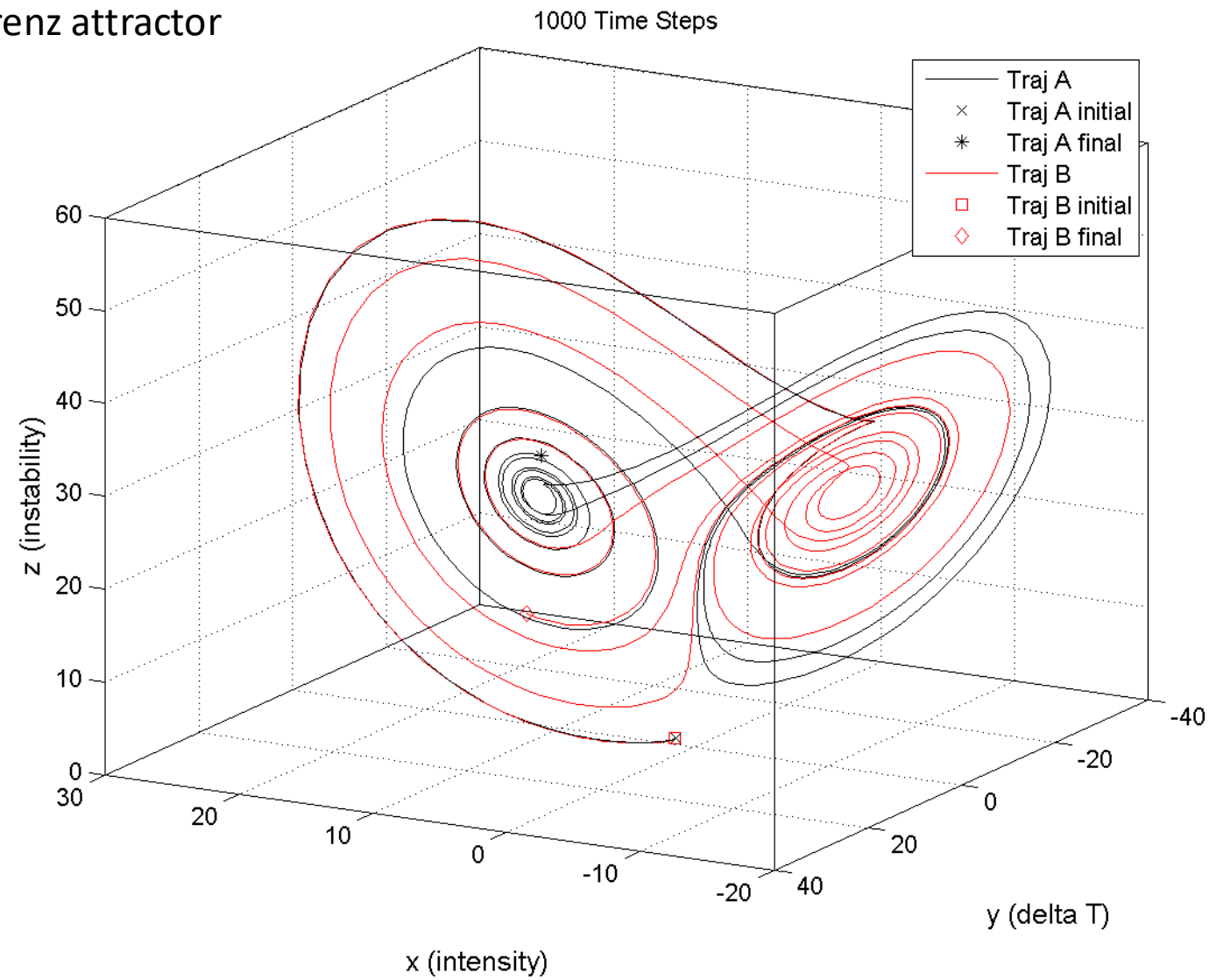
Lorenz attractor



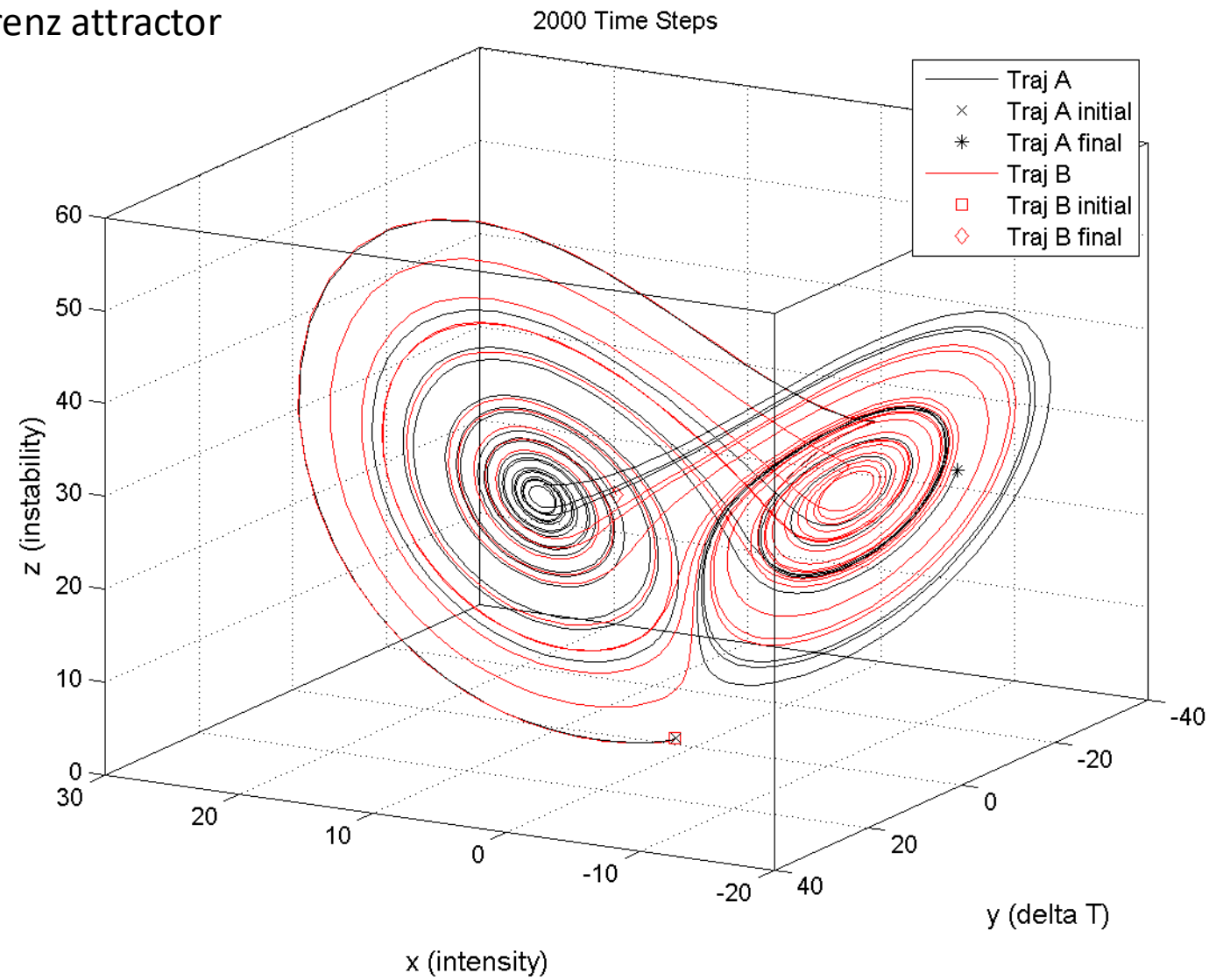
Lorenz attractor



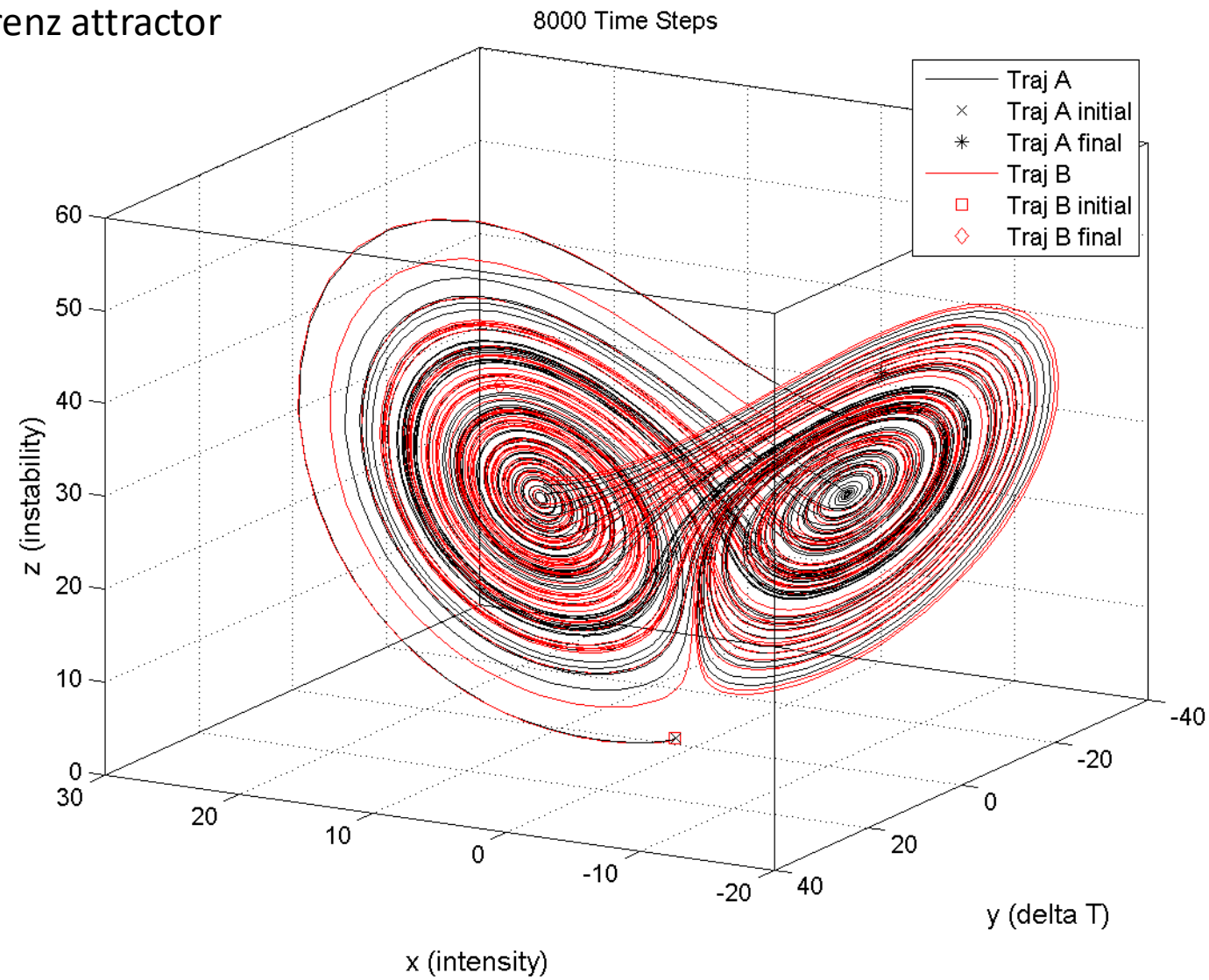
Lorenz attractor

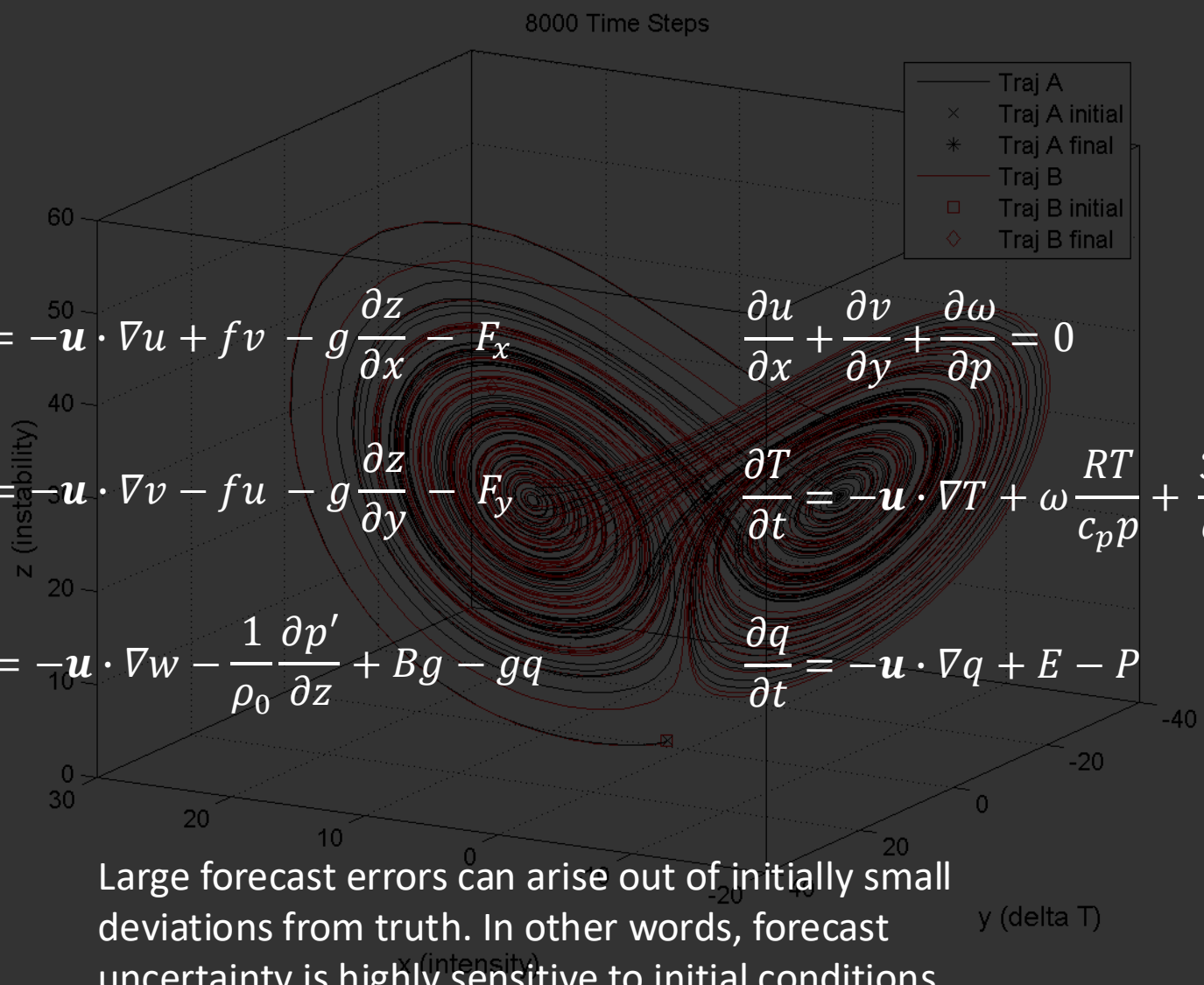


Lorenz attractor



Lorenz attractor





$$\frac{\partial u}{\partial t} = -\boldsymbol{u} \cdot \nabla u + fv - g\frac{\partial z}{\partial x} - F_x$$

$$\frac{\partial v}{\partial t} = -\boldsymbol{u} \cdot \nabla v - fu - g\frac{\partial z}{\partial y} - F_y$$

$$\frac{\partial w}{\partial t} = -\boldsymbol{u} \cdot \nabla w - \frac{1}{\rho_0}\frac{\partial p'}{\partial z} + Bg - gq$$

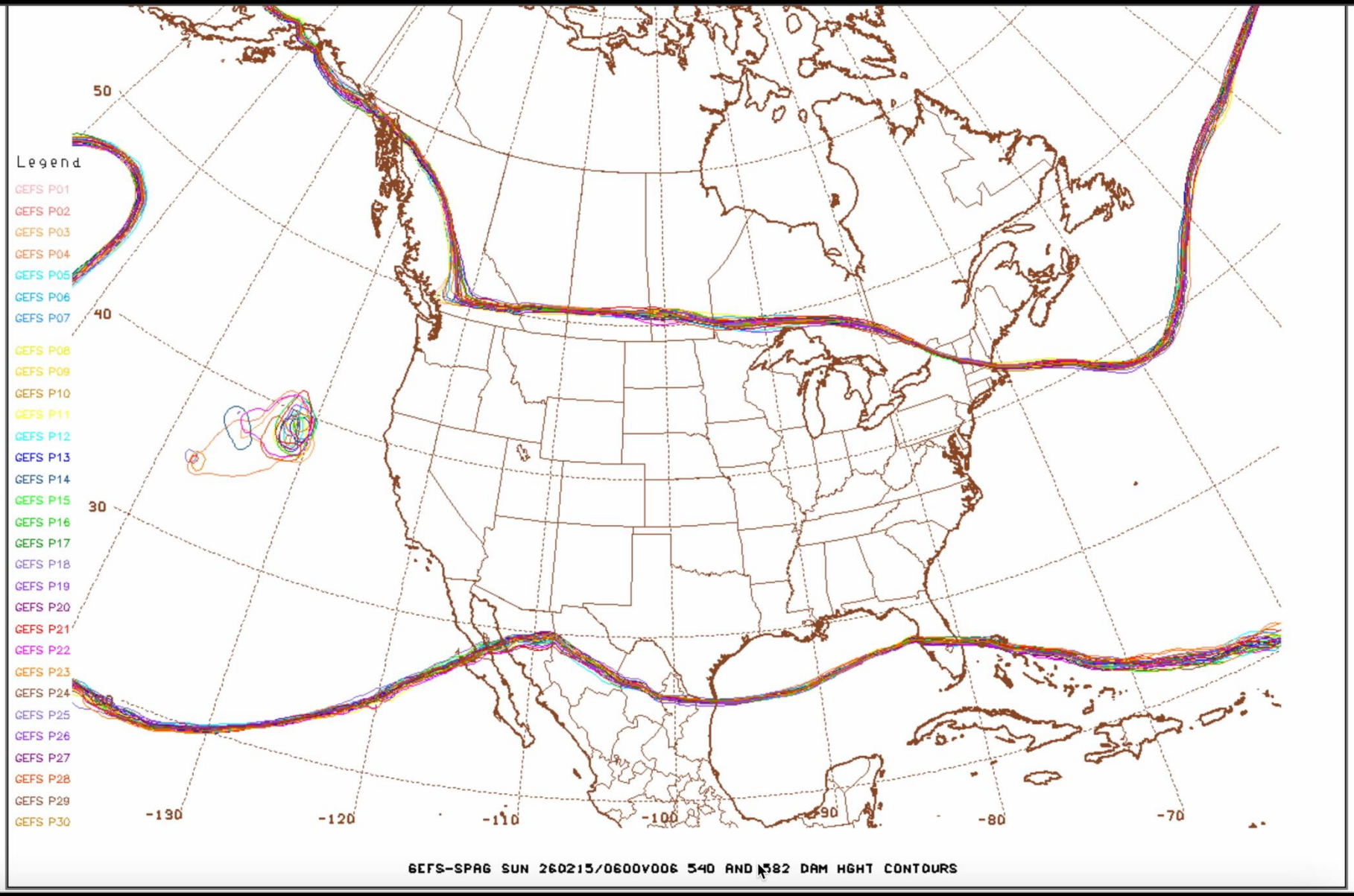
$$\frac{\partial u}{\partial x} + \frac{\partial v}{\partial y} + \frac{\partial \omega}{\partial p} = 0$$

$$\frac{\partial T}{\partial t} = -\boldsymbol{u} \cdot \nabla T + \omega\frac{RT}{c_p p} + \frac{\mathcal{H}}{c_p}$$

$$\frac{\partial q}{\partial t} = -\boldsymbol{u} \cdot \nabla q + E - P$$

Large forecast errors can arise out of initially small deviations from truth. In other words, forecast uncertainty is highly sensitive to initial conditions.

GEFS 540 and 582 dm contours



Major Global Atmospheric Models

	Deterministic	Ensemble
European	IFS	EPS (51 members)
American	GFS	GEFS (31 members)
Canadian	GEM	GEPS (21 members)

NWP ensemble systems re-run the same model many times on slightly different initial conditions → Computationally expensive!

ML ensemble systems can generate many members in parallel quickly. But how do they generate different forecasts?

The Naïve Way

Perturb initial conditions in some way and re-run inference.

- Gaussian noise, possibly targeted to certain areas?
- Use ICs from different ensemble members?

Problems:

1. How do you know how much noise to add and where?
2. Still have blurred ensemble members.
3. If using ICs from NWP ensembles, how do you know if uncertainty is properly captured?
4. Models were trained to predict on
   analysis → forecast,
   not analysis + noise → forecast.
   Therefore, noise is often dropped out quickly.

Often results in **underdispersed** ensembles. This is no better than NWP status quo.

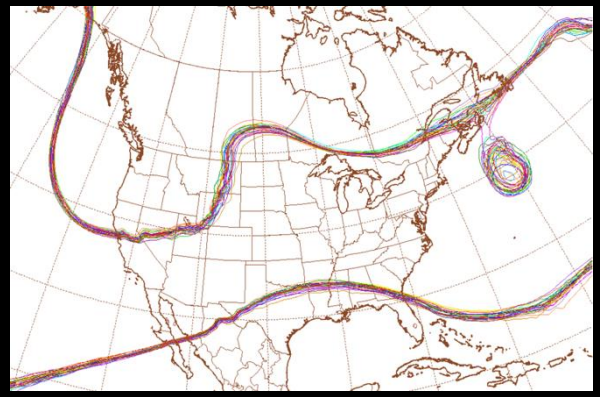
Ensemble Dispersion

Underdispersion: Model spread is less than observed, i.e., model is too certain. Common in NWP.

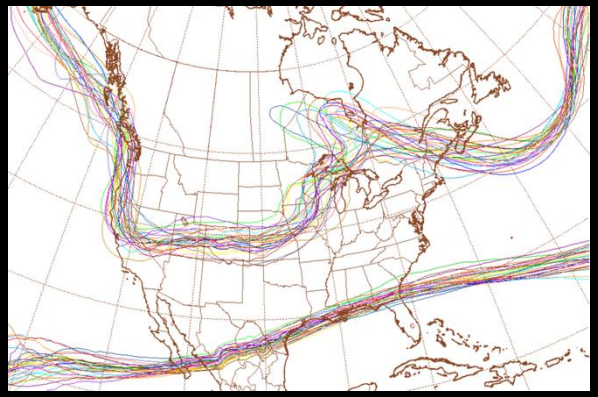
Overdispersion: Model spread is more than observed., i.e., model is too uncertain.

Well-calibrated: Model spread matches observed spread, i.e., model uncertainty is more trustworthy.

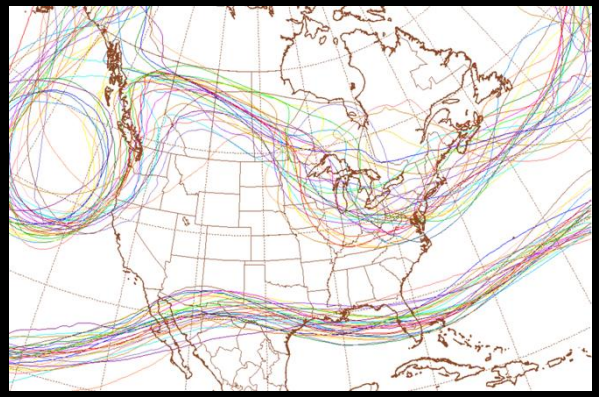
Underdispersed



Calibrated



Overdispersed

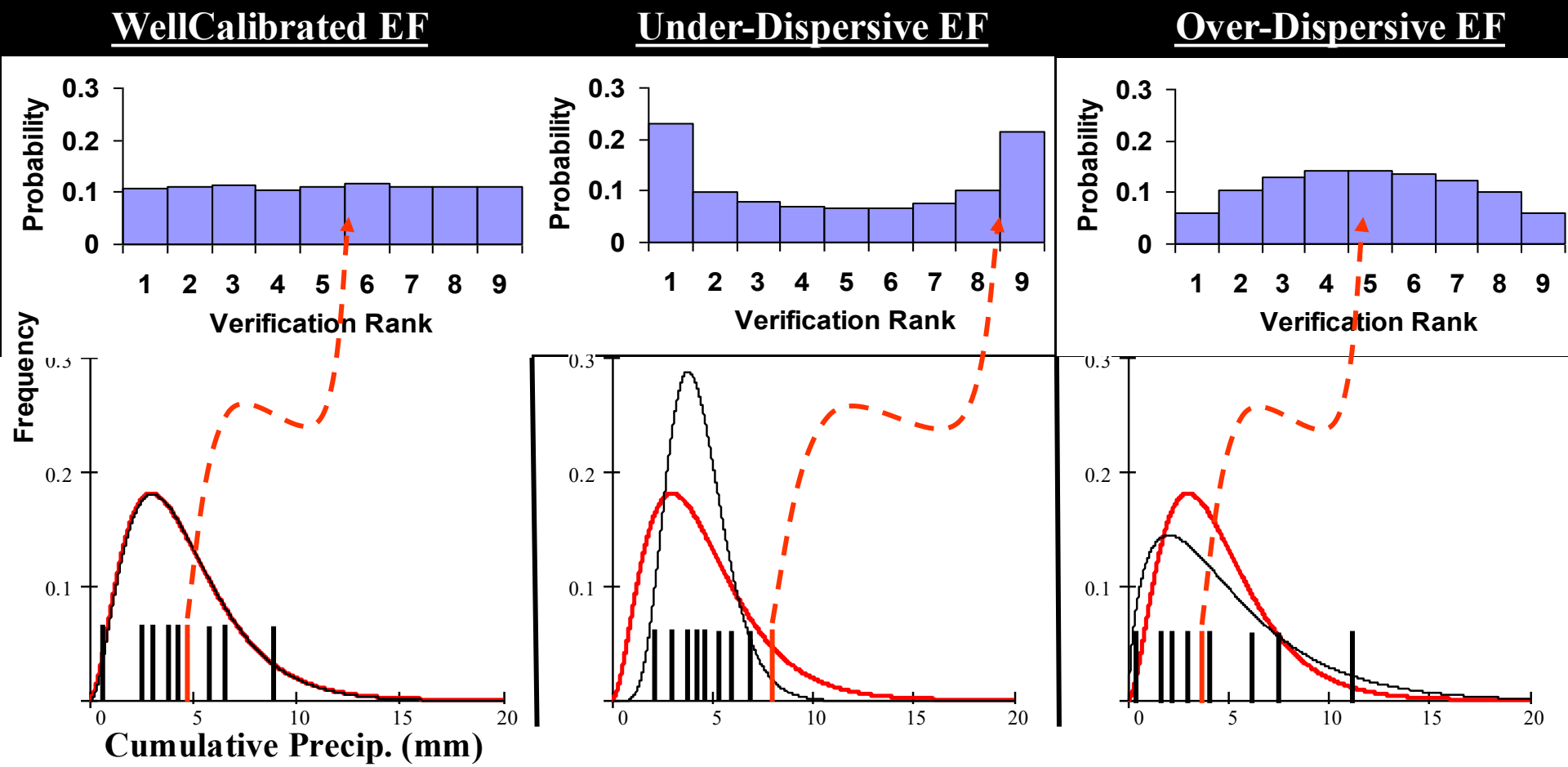


NOTE: These maps are from real GEFS forecasts at different forecast times. They may actually all represent well- calibrated forecasts for that forecast time. The point of this slide is to show what spread may look like at an arbitrary forecast time. However, well-calibrated spread increases with forecast uncertainty (e.g., at longer lead times).

Verification Rank Histogram

indirectly examines statistical consistency and dispersion

Over many trials, record verification's position (*the "rank"*) among the ordered EF members.

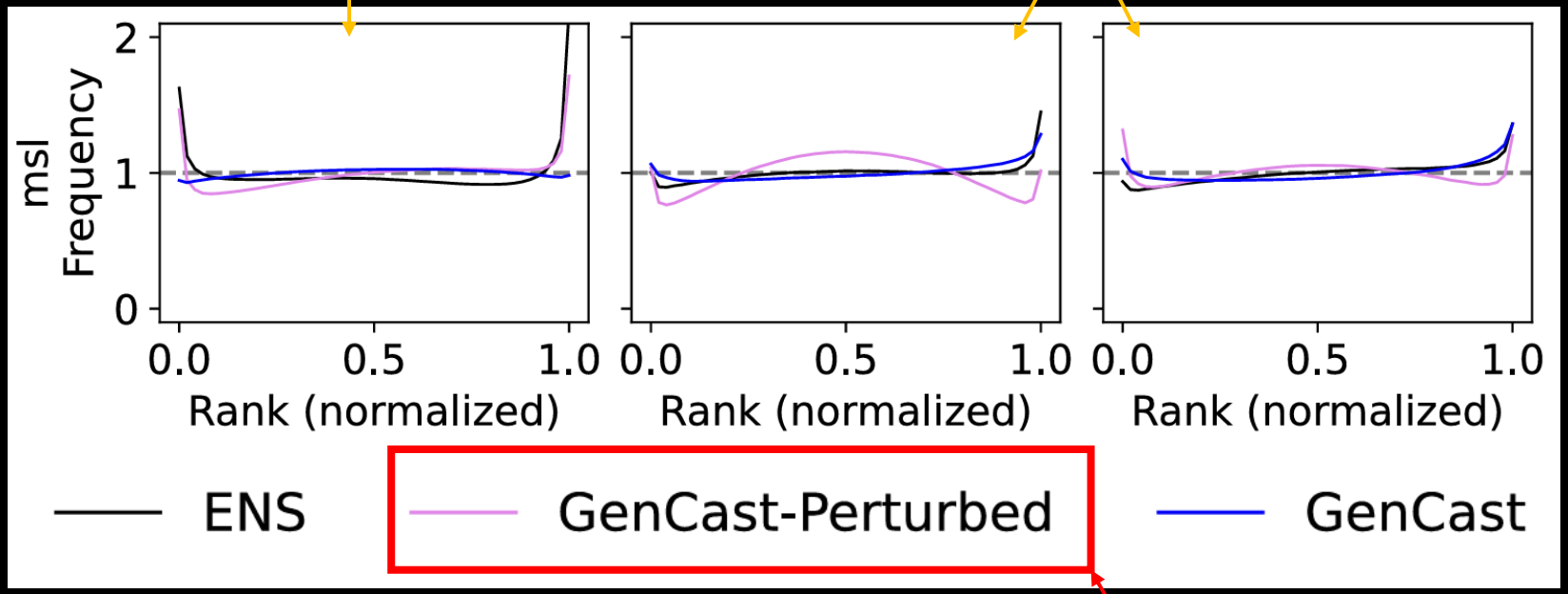


EF PDF curve & 8 sample members **True PDF curve & verification value**

Graphic adapted from presentation by USAF MAJ Tony Eckel, ret.

ENS underdispersed
GenCast better

Both ENS and GenCast
show similar dispersion:
underdispersed with low
bias



Price et al. (2025), supplemental info

Generally less reliable
dispersion than ENS or
GenCast

The Way

Train a model to learn probability distributions *conditioned on current and recent states*.

Google GenCast: A Conditional Diffusion Model

Conditional diffusion model

↓

Not physical diffusion!

Diffusion: Probabilistic denoiser model

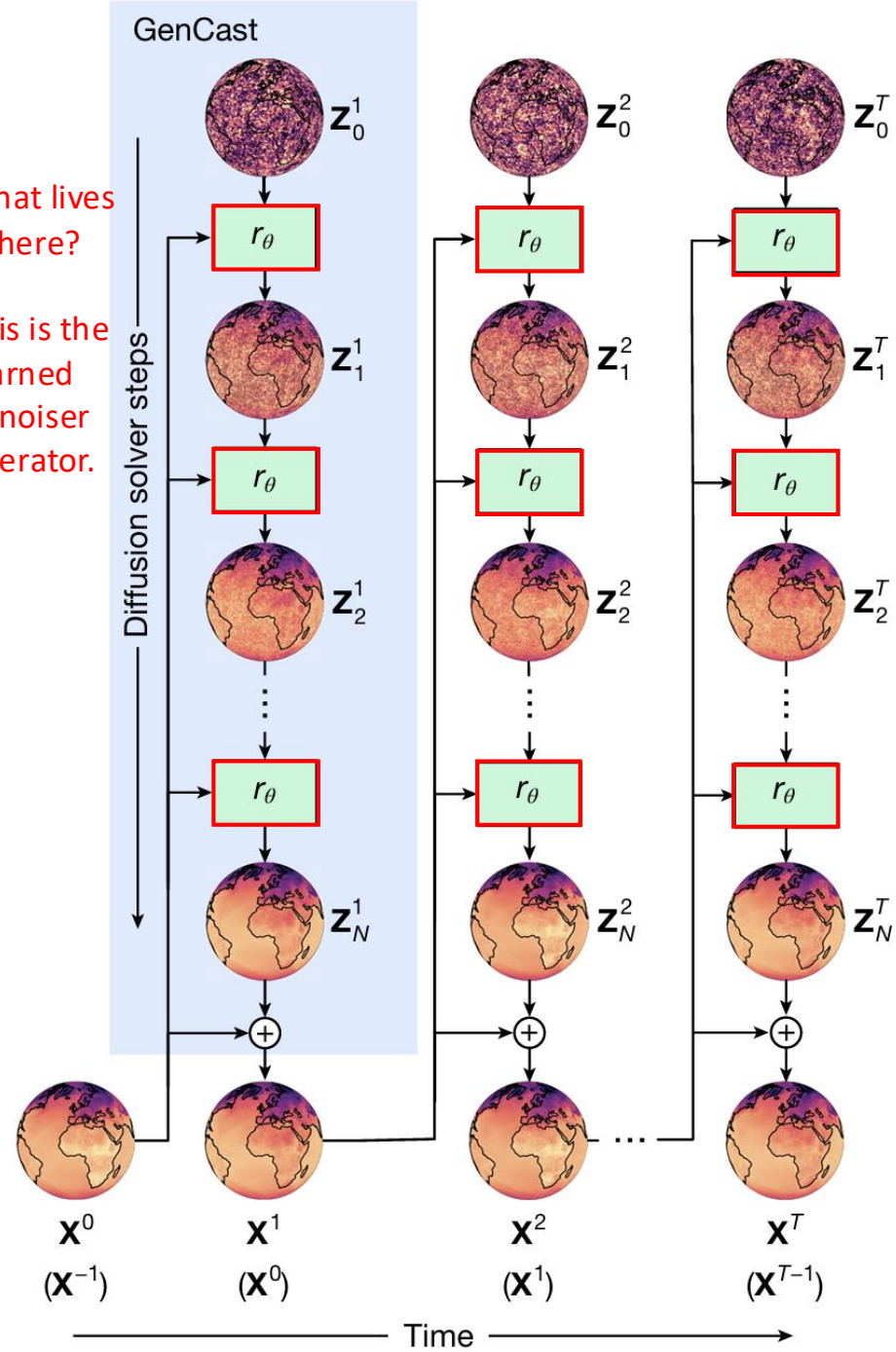
Conditional: Model is conditioned on past states

GenCast starts with white noise and tries to learn how to remove the noise so that it is left with a plausible solution for the forecast time.

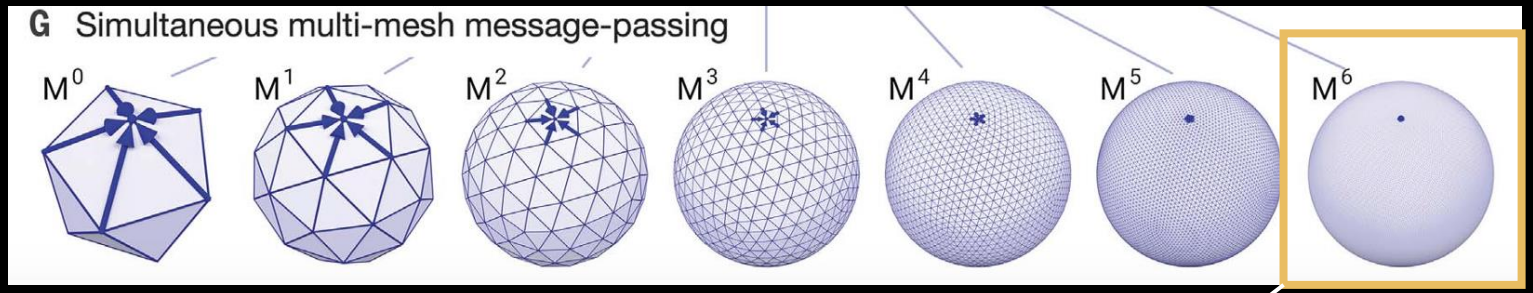
Since initial distribution is stochastic, each solution is different.

What lives in here?

This is the learned denoiser operator.



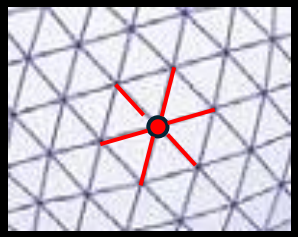
GenCast Architecture



Lam et al. (2023): GraphCast multi-mesh structure

GenCast uses the finest mesh only.

Zoom in

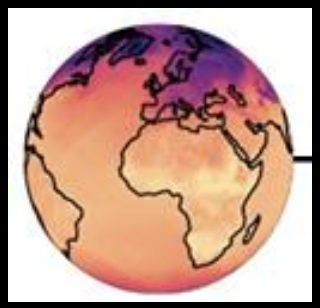


Graph-transformer:
Attention over one hop

Focus on *local* structure per iteration, but *regional* and *synoptic* structure is captured through successive transformer blocks (16 used during training).

Training

X_{-1}



X_{-2}

Condition on previous states

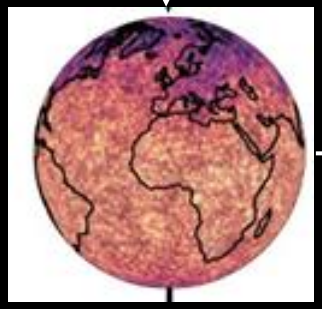
No autoregressive rollout!

Z_N

Training repeats this process for many different noise levels to produce a single operator.

$$Z^t = Z_N + \sigma_t \varepsilon$$

Z^t

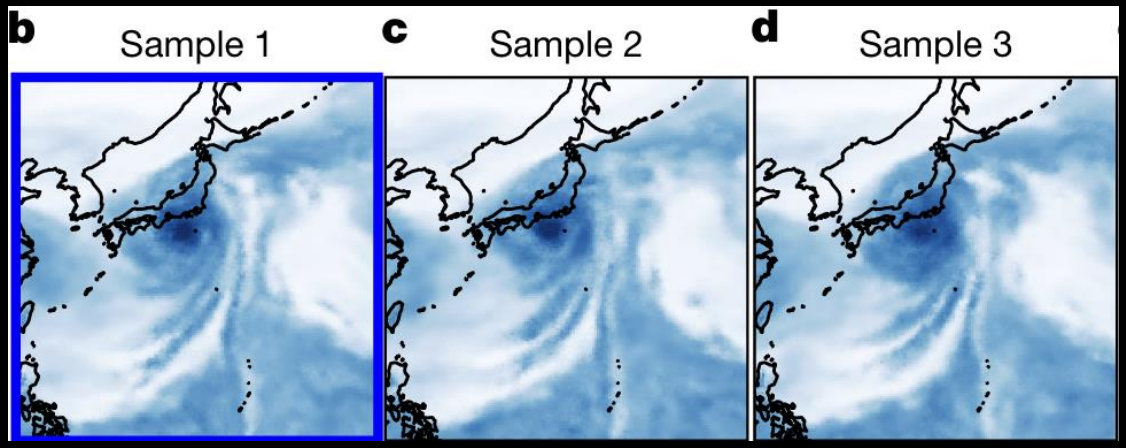


Attempt to denoise back to Z_0

Y^t

$$\sum_{t \in D_{\text{train}}} E\left[\lambda(\sigma)\frac{1}{|G||J|}\sum_{i \in G}\sum_{j \in J} w_j a_i \boxed{(Y^t_{i,j} - Z^t_{i,j})^2}\right],$$

Sharp forecasts!



$$\sum_{t \in D_{\text{train}}} E\left[\lambda(\sigma)\frac{1}{|G||J|}\sum_{i \in G}\sum_{j \in J} w_j a_i (Y_{i,j}^t - Z_{i,j}^t)^2\right],$$

$$Y_{i,j}^t - Z_{i,j}^t$$

is related to a ***score function***, i.e., GenCast *learns probability distributions*.

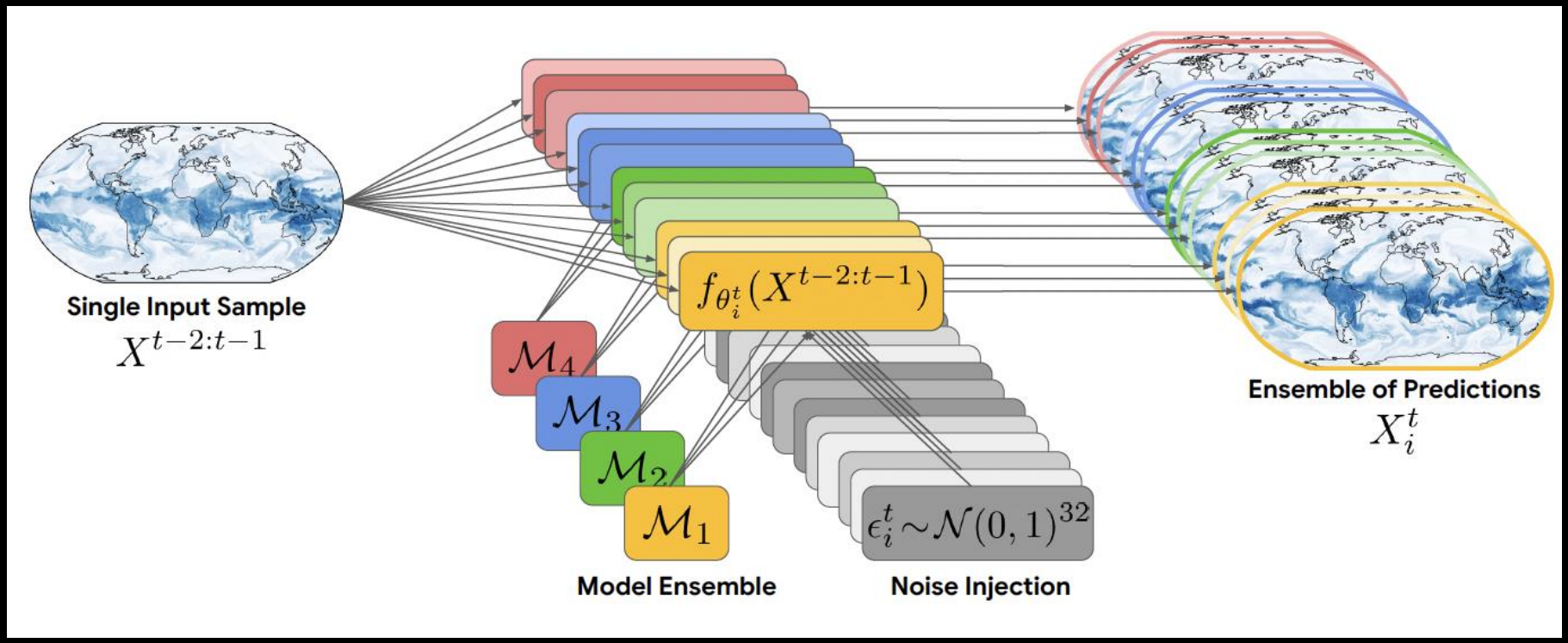
$$\nabla_{Y^t} \log p\left(Y^t | \sigma, X_{-1}, X_{-2}\right) = \frac{Y_{i,j}^t - Z_{i,j}^t}{\sigma_t^2}$$

How does this happen if we use MSE loss?

GenCast doesn't learn weights to move from current state to future state (which has many possibilities).

It learns how to denoise from a corrupted state to a single truth.

Functional Generative Network (FGN): Google WeatherNext-2


Single Input Sample
$X^{t-2:t-1}$
$f_{\theta_i^t}(X^{t-2:t-1})$
$\mathcal{M}_4$
$\mathcal{M}_3$
$\mathcal{M}_2$
$\mathcal{M}_1$
$\epsilon_i^t \sim \mathcal{N}(0,1)^{32}$
Model Ensemble
Noise Injection
Ensemble of Predictions
X_i^t

Learn probability of states:

$$p(X^{1:T}|X^0, X^{-1}) = \prod_{t=1}^{T} p(X^t|X^{t-2:t-1})$$

Minimize the skill-spread function:

$$\mathcal{L} := \frac{1}{|D|} \sum_d \frac{1}{G} \sum_i a_i \mathrm{fCRPS}(x_{i,d}^{1:2}, y_{i,d})$$

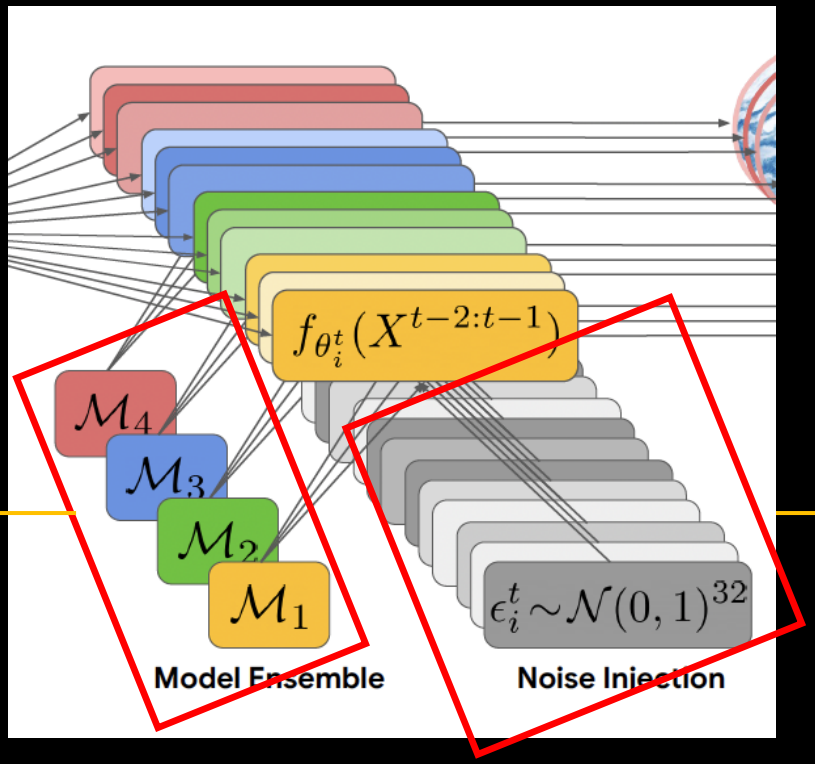
$$\mathrm{fCRPS}(x^{1:N}, y) := \frac{1}{N} \sum_n |x^n - y| - \frac{1}{2N(N-1)} \sum_{n,n'} |x^n - x^{n'}|.$$

Modeling Uncertainty

<u>Epistemic uncertainty</u>: Uncertainty that can theoretically be improved upon (e.g., observations).

<u>Aleatoric uncertainty</u>: Uncertainty associated with limitations in model (or in what can be modeled, i.e., stochasticity).

Epistemic



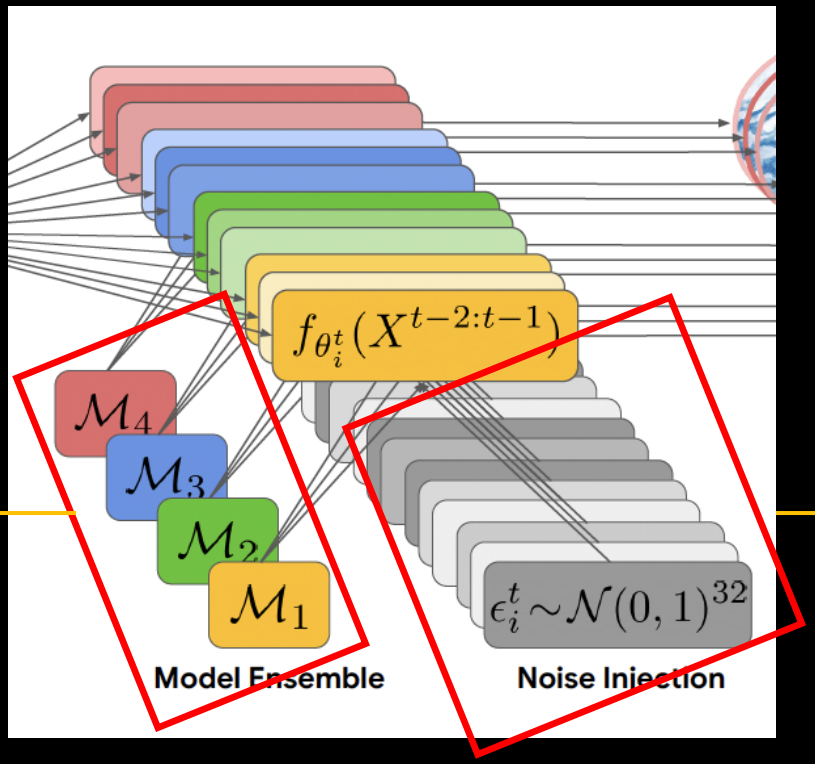
Aleatoric

Modeling Uncertainty

<u>Epistemic</u>: Each model *M* is separately learning a different set of weights for the same problem. This is called **deep ensembling** and improves spread.

<u>Aleatoric</u>: Noise is added in *parameter space*, not in latent space! This is the key difference from a diffusion model like GenCast.

Epistemic



Aleatoric

Injecting noise in Layer Normalization parameters

Layer Normalization happens inside a transformer block at each hidden layer and helps stabilize activation in deep ensembles.

Generally,

$$LN(h) = \beta + \gamma \frac{h - \mu}{\sqrt{\sigma^2 + \epsilon}}$$

μ: Mean of h
σ^2: Variance of *h*
ϵ: Small constant
β: Learned shift parameter: Offset
γ: Learned scale parameter: Amplification

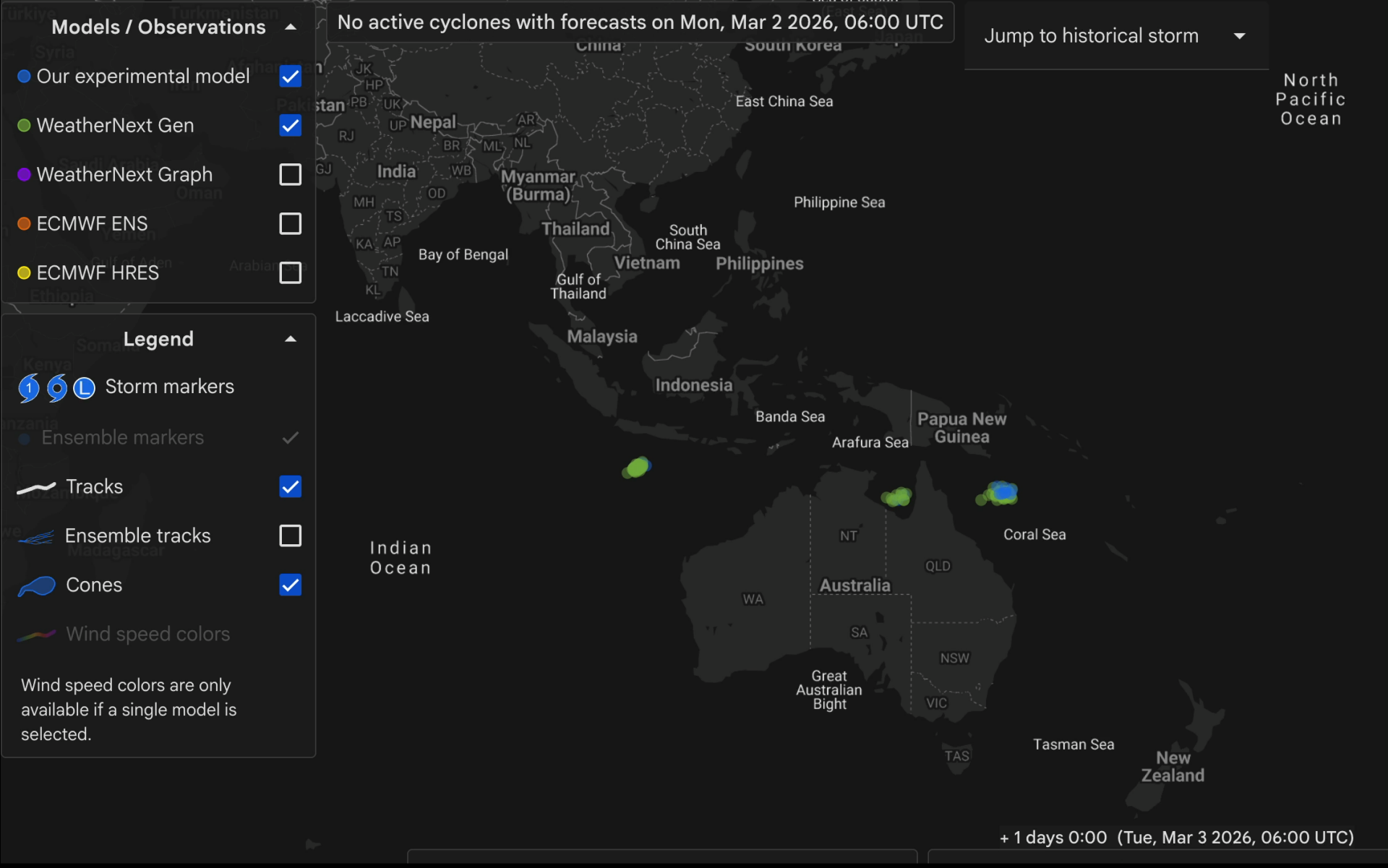
Noise modulates these!

At each layer, normalize to zero mean using variance of *h*. Scaling determines how much features propagate to next layer. Shifting impacts how much activation function is applied.

FGN vs. GenCast*

	FGN (Functional Generative Network)	GenCast
Probabilistic framing	Approximate Bayesian model averaging	Score-based diffusion modeling
Mathematical objective	Minimize CRPS	Minimize denoising score matching loss
Source of randomness	Latent vector (z) modulating LayerNorm	State-space Gaussian noise at multiple σ levels
Noise Introduction	Parameter space (via normalization modulation)	State space (additive noise to atmospheric fields)
Conditioning mechanism	Conditional LayerNorm on latent vector	Conditional LayerNorm on diffusion noise level
Autoregressive rollout	Yes	Yes
Epistemic uncertainty	Deep ensembles (multiple trained models)	Usually single model (can ensemble seeds)
Aleatoric uncertainty	Latent noise vector	Diffusion sampling
Inference	Single forward pass per timestep	Multiple denoising steps per timestep (ODE solve)
Ensemble generation cost	Linear in number of members	Linear × number of diffusion steps
Computational cost	Much lower at inference	Higher due to iterative sampling

**Modified ChatGPT summary*



https://deepmind.google.com/science/weatherlab